ChatGPT

THE PRODUCT: Technical & Architectural Blueprint

Architecture Overview (Browser Extension + Backend)

Hybrid Extension + Backend Design: Yes – a hybrid model with a Chrome extension frontend and a dedicated backend (e.g. Flask server with a database) is ideal. The Chrome extension will integrate directly into the Faceit matchroom webpage to display data, while heavy processing (data collection, demo parsing, analytics, user auth) runs on backend servers. This separation is necessary because parsing CS2 demo files and aggregating stats is too intensive to perform in-browser. In fact, uncompressed CS2 demos can be hundreds of MBs in size (up to ~500MB) and require significant processing per file [1]. Running such analysis in the extension (browser) would be impractical due to download and execution time [2]. Instead, the extension acts as a lightweight client: it detects the match context (matchroom URL, participating player names/IDs) and sends a request to the backend for precomputed insights. The backend (built with Python Flask, given the developer's familiarity) will handle data retrieval (from Faceit APIs or scraped sources), demo parsing, and analytics, then respond with summarized insights that the extension injects into the Faceit UI.

Communication Flow: When the user joins a Faceit matchroom (during the map veto phase), the content script of the extension will scrape the necessary identifiers (match ID and player nicknames/IDs from the DOM). It will then call the backend's REST API (e.g. `GET /prematch-info?match=<ID>` or send the list of player IDs) over HTTPS. The backend, in turn, gathers all required data (player stats, recent match demos, parsed analytics) and returns a JSON payload of insights. The extension receives this data and dynamically injects UI elements into the Faceit page to display the information (e.g. beside player names or in a sidebar panel). This architecture is similar to how existing Faceit extensions augment match pages with external data – for example, the Repeek (Faceit Enhancer) extension calls external APIs to show team Elo ratings in match rooms [3]. By offloading data processing to the backend, we ensure the extension remains fast and lightweight, only handling UI rendering and simple logic.

Backend Components: A Python Flask server is suitable for the web service, exposing endpoints to the extension. MongoDB is a good choice for the database due to its flexibility in storing complex structured stats per player (e.g. nested JSON for maps, weapons, positions, etc.) that come from parsing [4] [5]. The developer's familiarity with Python, Flask, and MongoDB makes this stack a natural fit. The backend will also include background workers for intensive tasks (discussed later), and possibly a small authentication service if a freemium login model is used. Overall, this client-server split aligns with the developer's skillset and ensures that **THE PRODUCT** can perform heavy data crunching server-side while seamlessly integrating into the user's browser experience.

Data Sourcing Strategy (Faceit Stats & Demo Retrieval)

Faceit Data API: The best approach to source player and match data is to leverage Faceit's official Data API. Faceit provides REST endpoints to retrieve player details, statistics, match history, and even downloadable demo URLs [6]. Using the API (with a developer API key), the backend can programmatically fetch: - **Player**

Profile and Stats: For each player in the match, query the Faceit Data API for their profile and lifetime stats (e.g. skill level, ELO, overall performance). Notably, the API's **"Retrieve statistics of a player"** endpoint returns per-map statistics (matches played, wins, win rate %, K/D, etc.) under a **"segments"** array [4] [5]. This means we can directly obtain each player's win rate (%) on each map and number of matches on that map without parsing demos. For example, the API might show a player has a **53% Win Rate** on *de_cache* across 78 matches [7]. Such data is invaluable for map veto decisions (and can be displayed instantly via the extension). - **Match History & Demo URLs:** Using the **"Retrieve all matches of a player"** endpoint, the backend can list recent matches for each player. Each match detail can then be fetched via **"Retrieve match details"**, which includes an array of `demo_url` links [6] for the replay files. With a player's recent match IDs (or the current match ID if known), the system can gather the URLs to download the actual `.dem` replay files for analysis. Faceit's API typically provides direct links (often to cloud storage) for demo downloads. This is how services like Scope.gg's Prematch tool are able to download the last 5 matches of opponents automatically [8]. Since these demo files are public for anyone with the link (Faceit allows any user to download recent match replays of other players [9]), our backend can retrieve them without violating terms.

Open-Source Tools & Scraping: In addition to the official API, there are community tools that can aid data collection: - *Faceit Scraping Libraries:* If the API has strict rate limits or missing info, an alternative is to scrape Faceit web pages (e.g. player profile pages or match pages). However, given that **all needed data (player stats, match history, demo links)** are available via the official API [10] [6], pure scraping is usually not necessary and can be brittle. It's more robust to use the documented API with proper API key management and caching. For instance, the **Faceit Analyser** and **VisusGG** tools rely on Faceit's API to get match histories and stats (VisusGG then displays map win rates for each player during map selection) [11]. - *Community APIs/ Projects:* Some open-source projects (like the **Lookback-FACEIT** GitHub project or Faceit Analyser API) demonstrate how to query Faceit data for matches and players. These can serve as references or even be leveraged for certain features. For example, Lookback-FACEIT shows how to page through a user's match history using the API's offset and time filters (useful for gathering many past matches) [12]. However, for our purposes – analyzing recent performance – it's usually sufficient to fetch the last X matches for each player via the official endpoints. - *Demo Sources:* Besides Faceit, CS2 demos can sometimes be found on other platforms (e.g. professional matches on HLTV or community sites like **cs2stats**). Since **THE PRODUCT** focuses on Faceit pug matches, we will primarily stick to Faceit's own data. Every Faceit match ID yields a demo file link via the API, so we do not need external sources for demos.

Data Gathering Workflow: For each match that the extension needs to analyze: 1. **Identify Players:** The extension sends the list of player nicknames or IDs to the backend. If only nicknames are available from the UI, the backend will first resolve each nickname to a Faceit `player_id` (via the `/players?nickname=` search endpoint or by storing a mapping from previous lookups). 2. **Fetch Player Stats:** Immediately retrieve each player's lifetime or recent stats from the Faceit API. This provides quick info like win rates on maps, average K/D, HS%, etc. (to be displayed as high-level stats). For example, obtaining map win percentages and number of matches can directly inform which maps the opponents are weakest on. 3. **Fetch Recent Matches:** Using each player's `player_id`, fetch their recent match IDs (e.g. last 5-10 matches). Optionally filter for competitive CS2 matches (5v5) if the API returns other queue types. This step yields a set of match IDs that cover the recent history of all players. 4. **Download Demo Files:** For a selection of those match IDs (e.g. the last 5 matches for each opponent, as in Scope Prematch [13]), request the `demo_url` from the match details and download the .dem files to the backend server (likely storing them in a temporary directory or in memory). If a demo was already recently downloaded (cached) for a given match or player, skip re-downloading to save time. 5. **Leverage Public Stats:** In parallel with demo

downloading, gather any additional public stats that don't require parsing – e.g. team Elo ratings or Faceit levels (the extension could display team Elo difference, which Faceit Enhancer does by fetching each player's Elo externally [3]). Faceit API's player detail can provide current Elo and rank. These quick stats can be sent to the extension immediately while deeper demo analysis continues in the background.

By using the official API and caching aggressively, the system minimizes direct scraping. All **insights are derived from public data** that any user could manually obtain (Faceit profiles, match pages, and demo files), just automated – so this approach abides by Faceit's rules (Scope.gg explicitly notes that prematch analytics only use publicly available data and thus are safe [9]). In summary, the backend will **serve as a data aggregator**, pulling Faceit stats and demos on demand. Open-source Faceit libraries or examples can jump-start integration, but the core data pipeline will revolve around Faceit's Data API for reliability.

Demo Parsing Engine (High-Performance Parsing of .dem Files)

Choosing a Demo Parsing Library: To extract deep insights from CS2 demo files, we need a robust and fast parsing library. The leading choice is **demoinfocs-golang**, a Go library known to be *"blazing fast, feature complete and production ready"* for CS:GO/CS2 demo parsing [14]. This library was built with performance and concurrency in mind and can parse large demo files very efficiently. Benchmarks indicate it can parse ~25 minutes of gameplay per second on a single thread [15], meaning a full match demo (typically ~30 rounds, ~45-60 minutes in-game time) can be processed in around 1–2 seconds in ideal conditions. It also supports parsing multiple demos in parallel threads, scaling with CPU cores [15]. This level of performance is crucial for **THE PRODUCT**, which may need to parse ~10–25 demos within a short pre-match window.

Industry-Proven Solution: Demoinfocs is widely trusted – it's used by major CS analytics services and sites (including HLTV, Scope.gg, and others) [16]. By adopting it, we stand on the shoulders of proven technology for accurate event parsing (kills, grenades, player positions, etc.). The library is updated for Counter-Strike 2 (v4 branch for CS2), keeping pace with any changes in demo format.

Integration Approach: Since the developer is more familiar with Python/C++ than Go, there are a couple of ways to integrate a Go-based library: - *Use a Python Wrapper:* The **"awpy"** library is a Python toolkit that wraps around CS:GO/CS2 demo parsing functionality (initially built on demoinfocs) [17]. Awpy (and its underlying `demoparser2`) allows you to parse demo files and get structured output in Python directly, without writing Go code. This could be the fastest path to integration – the developer can `pip install awpy` and call its API to parse a demo file, receiving events, player positions, kills, etc., as Python objects. Awpy was specifically created to bring high-performance demo parsing to Python users, and is maintained as CS2 evolves [18]. - *Standalone Parser Service:* Alternatively, run demoinfocs as a separate service. For example, implement a small Go program (or use existing CLI tools from the demoinfocs repo) that reads a .dem file and outputs the desired summary (in JSON). The Flask backend could invoke this parser via command line or RPC. However, this adds complexity (managing an external process or service) and inter-process communication overhead. Given the solo-developer context, a simpler in-process solution is preferable if performance allows it.

Valve's Official Parser: Valve provides a demo parsing tool (**demoinfogo**) in their CS:GO SDK [19]. While it's the reference implementation, it's essentially a command-line program and not as easy to use as a library. It's useful for verifying parsing accuracy, but many community projects (like demoinfocs) have built more accessible libraries on top of Valve's work [20]. We can credit Valve's parser as an inspiration (demoinfocs is

partly based on it [20]), but for scalable performance and ease of development, demoinfocs/awpy is the better choice.

Scalability and Performance Considerations: Using demoinfocs (directly or via awpy) will ensure that parsing multiple demos concurrently is feasible in a short time frame. For example, parsing 5 demos concurrently might take on the order of a few seconds total on a multi-core server [15]. This means even analyzing all 10 players' recent matches (~25 demos) could complete in well under a minute with enough parallelism. The library also provides granular control – we can choose to parse only specific events or frames if we want to speed up analysis by ignoring unneeded data (e.g. skip chat, or only track certain rounds). In practice, we will likely parse each demo fully but filter out only the data we need for our insights to minimize overhead.

Conclusion (Library Choice): Demoinfocs-golang is the recommended parsing engine for its speed and accuracy. The developer can integrate it via the **awpy** Python library [17] to stay within a comfortable tech stack. This combination yields a scalable solution: capable of parsing many demos quickly and extracting all the rich data (kills, positions, grenade throws, etc.) needed for building tactical insights.

Pre-Match Data Insights (What to Analyze and Deliver)

With data in hand (from Faceit stats and parsed demos), **THE PRODUCT** will generate *actionable tactical insights* for the match. We must focus on information that is **realistically obtainable** from available data and immediately useful during the matchroom (veto) stage. Below are the key data types and how they'll be derived:

- **Map Veto Stats (Win Rates & Pick/Ban Tendencies):** Using Faceit's lifetime stats and recent match history, we can show each team's and each player's performance on all competitive maps. Specifically, for each map we can display the team's win percentage and how often they play or ban that map. For example: *"Mirage – 72% win (18 plays)"*, *"Ancient – 40% win (5 plays)"*. The extension can highlight maps where the opposing team is weak. *Pick Rate (%)* can be inferred from how frequently a map appears in their last N matches. This helps the user make informed veto decisions. **VisusGG** (a similar extension) already demonstrates the value here by displaying opponents' win percentages per map during map selection [11]. Such stats allow players to identify opponent weaknesses and influence which maps to pick or ban [11]. Since these numbers come directly from Faceit stats (or quick aggregation of recent matches), they can be computed almost instantly and shown as soon as the matchroom is joined.

- **Player Playstyle Profiling:** By parsing recent demos, we can offer a breakdown of each opponent's playstyle on the soon-to-be-played map (or generally). This includes:

- *Common Positions (CT Side Heatmaps):* Identify where each player usually plays on CT side. For instance, *"PlayerX anchors B site 80% of rounds on Mirage"* or *"PlayerY often plays A-site rotate/ connector"*. We can derive this by tracking the positions where the player spends most of each round or where they get most of their kills on CT. This could be visualized as a simple heatmap on the map image or summarized in text form (e.g. *"frequent spots: window and catwalk"*). These **CT-side heatmaps** can warn our users which bombsites or angles a given opponent is likely to hold [13].

- *Aggression & Push Habits (T Side & CT Side):* Determine how aggressive a player is. For CT side, find if they push through smokes or flank often (e.g. *"PlayerZ frequently pushes mid early on CT side – beware of aggressive flanks"*). For T side, see if a player is an entry fragger or lurker. This can be quantified by looking at who gets first kills on T side, how quickly they make contact with the enemy, or if they consistently hit one bombsite. For example, *"PlayerX tends to rush B apps on pistol rounds"* or *"PlayerY is a slow lurker (often last alive on T)"*. These insights come from positional and event data in demos (timing of first engagements, routes taken).
- *Utility Usage Patterns:* Analyze each player's grenade usage. For example, how many flashbangs or smokes they throw on average, and any signature grenade spots. *"PlayerA throws ~2 flashbangs per round (above average), often flashing mid for teammates"* or *"PlayerB consistently molotovs A ramp at round start on CT"*. Such trends can be parsed from demo event streams (each grenade throw event includes thrower and grenade type). This tells our user which opponents are utility heavy and what to expect (so they can, say, be ready to dodge a flash or know a smoke is coming). If a player rarely uses utility, that's also notable (they might rely on aim duels more).

- *Preferred Weapons & Performance:* From stats or parsing, note if a player is an AWPer or rifle user predominantly, and how effective they are. E.g. *"PlayerC is the dedicated AWPer (uses AWP 30% of kills)"* or *"PlayerD has a high headshot rate with rifles (60%+ HS)"*. This can hint at how to counter them (avoid peek angles where the AWPer plays, etc.). Some of this data is directly in Faceit stats (headshot %, K/D) and can be supplemented with demo analysis for weapon usage per round.

- **Team-Level Trends:** Combining players' data, the system can also output team-wide insights:

- *Strong/Weak Maps:* Identify which map the opposing team (as a group of five) is worst at. This is extrapolated from individual win rates and the overlap of maps they have low success on. If, say, multiple opponents have low win rates on **Nuke**, one can assume the team overall would underperform on Nuke. This informs the user's team which map to aim for in the veto.
- *Preferred Sides or Starts:* Maybe the data shows the opponents tend to have much higher win rates on one side of a certain map (e.g. they always start CT on Overpass and have strong CT rounds). This could be shared to manage expectations (if we end up T first, it might be a rough start).
- *Recent Form:* Using recent match outcomes from Faceit, we can indicate if a player is on a winning streak or slump. For example, *"Opponent team has lost 4 of their last 5 matches"*. While not deeply tactical, it gives context about momentum.

Importantly, **all these insights are realistically obtainable** either from Faceit's provided stats or by parsing a handful of recent demos per player. This is exactly the kind of analysis professional tools do: Scope.gg's Prematch analyzes the last 5 matches of each opponent and shows *"favorite positions and zones of aggression on CT side, playstyle and preferred weapons"* [8]. We aim to deliver similar insights, focused on the map being played and the key tendencies of each opponent. The extension's job is to condense these into quick-read summaries or visual aids during the matchroom phase.

Contextual Delivery: During **map veto stage**, the extension will emphasize map-related stats (to assist in ban/pick decisions). For instance, as each map is being considered, it could display the opponents' win rates on that map next to its name (perhaps highlighting the lowest win rate in red as a favorable pick for our side). Once the map is decided, the extension can switch to showing player-specific intel for that map – e.g. "Opponents on Mirage: PlayerX=B apps player, PlayerY=mid AWPer, etc." – so the user can mentally prepare or discuss with teammates in the remaining time. All of this data is precomputed or quickly computed from the sources mentioned.

In summary, **THE PRODUCT** will parse and serve data such as: per-map win percentages (from Faceit stats) [7], favored positions and aggression zones (from demo heatmaps) [13], typical attacking routes, and utility usage habits. This is grounded in what's realistically attainable and **immediately useful** to high-tier players looking for any edge before the match. The data will be presented in a concise, actionable manner (not raw numbers overload, but meaningful insights like "Team tends to avoid Dust2 – likely ban from them" or "PlayerZ often pushes through smoke – be ready"). All insights derive from players' actual match history, giving our users a tactical dossier on their opponents within seconds.

Performance & Scalability Considerations

Handling Large Demo Files Quickly: CS2 demo files are quite large (dozens to hundreds of MB), and parsing them involves reading thousands of game frames. To provide **fast analytics during the short matchroom phase (a couple of minutes)**, the backend must manage these files efficiently: - *Parallel Processing:* We will parse multiple demos in parallel to utilize multiple CPU cores. Demoinfocs is thread-safe and supports concurrency [15]. For example, if we need to parse 20 demo files and our server has 8 cores, we can spin up 8 parsing routines to work simultaneously – achieving an effective throughput much higher than linear parsing. The goal is to crunch through all required demos within e.g. 30 seconds or less. Benchmarks show ~0.9s to parse a 25-minute demo on one core [15], so with parallelism this is feasible. We will have to be mindful of memory (loading too many demos at once can consume several GB of RAM, given a 300MB demo can expand when parsed) and adjust concurrency accordingly. But in general, concurrency will be leveraged to meet real-time demands.

- *Selective Parsing & Precomputation:* We don't always need to parse *every* recent demo at match time. **Precomputation and caching** will greatly improve responsiveness. We can maintain a database of parsed results for players (or matches) updated periodically:
- When a player's new match is finished (or when we first encounter a player), a background job can download and parse that demo, then store the extracted stats (kills, positions, etc.) in our database. Then at match time, if the same demo or player is needed, we already have their data ready to go. This concept is akin to caching the "events" extracted from demos so we don't reparse the same file repeatedly [21] [22]. Andrew Healey's analysis notes that *"parsing a 300MB demo file takes ~20sec on a desktop PC"* and suggests the obvious optimization: **store the generated events so you don't need to reparse every time** [21] [23].
- In practice, this means if many high-tier players use the product regularly, we'll accumulate a repository of recent match analytics. The backend can check: *Do we already have data on player X's last 5 games?* If yes, fetch from DB and avoid re-downloading/parsing those demos. If a demo for match Y was parsed for another player's analysis, reuse that data for all players in that match.

- Even within a single match's context, note that both teams' players might have overlapping history. We should avoid duplicate downloads/parses. For example, if two opponents played in the same recent match, that one demo yields data for both. Our system can parse it once and attribute stats to all relevant players.

- *Real-Time vs Offline Analysis:* For an MVP, it's acceptable to perform some parsing on-demand (when a matchroom is joined) as long as it finishes in time. However, to guarantee speed, the strategy leans on **offline pre-processing**: regularly pulling recent matches for active players and computing their insight profiles. A scheduled job could periodically update a "player insight cache" for top Faceit players (especially in high Elo where our target users play). Then, when our user enters a match,

most of the heavy lifting is already done. This design is akin to building a mini data warehouse of player stats from demos. The backend can then serve queries like *"give me player X's Mirage heatmap and stats"* from the database almost instantly, rather than computing it from scratch.

- *Fast Data Retrieval:* We will use indexing and efficient queries on the backend to retrieve the precomputed data. For example, each player could have a document in MongoDB containing aggregated stats per map, favorite positions, etc. Fetching 10 player records from Mongo is trivial (a few milliseconds). If some data isn't precomputed, we could fall back to on-the-fly parsing, but the UI might show a loading indicator for those pieces. In the worst case where no data is cached and we must parse everything fresh, we might deliver initial basic stats (map win rates) immediately and then update with deeper insights as they complete parsing. Given the matchroom lasts maybe 60-90 seconds until the game launches, any parse tasks should aim to finish within that window.

- **Scaling with User Load:** As a solo developer, the initial user base might be small, but planning for growth is important:

- The backend should be able to handle multiple simultaneous requests (i.e. multiple matches being analyzed). This means potentially parsing many demos concurrently across requests. A task queue system (like Celery or a custom thread pool) can queue parsing jobs and distribute them so that the server isn't overwhelmed. If demand grows, the parsing workers could be scaled out to multiple machines or containers.
- Rate limiting Faceit API calls is also important – we should cache API responses (e.g. a player's lifetime stats can be cached for hours, their recent matches for a few minutes) to avoid hitting the API limits when many users request data. If the API limit is reached, having a fallback (like delaying non-critical data or using a secondary key) might be necessary.
- We'll design the system such that **premium features** (like very detailed heatmaps or analysis of more than last 5 demos) are only computed for paying users, to control computational load. Freemium users might get the basics (which are lighter to compute) and see an upgrade prompt for deeper analysis.

Hosting and Infrastructure (discussed next) will also contribute to performance, e.g. choosing a server with SSD storage (faster demo file I/O) and ample CPU/RAM for parsing bursts. By using concurrency, caching, and careful resource management, the backend can deliver the needed analytics in near-real-time. The overarching principle is to **avoid redundant work**: download and parse only what's necessary, and reuse results wherever possible [22]. With these optimizations, even the intensive task of parsing large CS2 demos becomes manageable within the matchroom timeframe.

Hosting & Infrastructure Strategy

For a solo-developed product, the hosting solution must balance **cost, simplicity, and scalability**. Here are the recommendations:

- **Cloud VM or VPS (Initial Deployment):** Start with a general-purpose cloud server (VPS) hosting both the backend application and the database. For example, a DigitalOcean droplet or AWS Lightsail instance with sufficient CPU (4+ vCPUs) and RAM (8–16 GB) would allow running the Flask API, MongoDB, and parsing workers all together. This option is cost-effective and straightforward to manage via Docker or similar tools. The developer's familiarity with Docker means the whole stack

(Flask app, MongoDB, perhaps a Redis for task queue) can be containerized and run on one host. A single server setup is acceptable for initial MVP traffic and keeps latency low (internal communication is all on the same machine).

- **Scalability and Cloud Services:** As the user base grows or parsing demands increase, consider a more scalable cloud architecture:

- *Separate Services:* Split the database onto a managed cloud DB service (e.g. MongoDB Atlas or AWS DocumentDB) so it can handle load and be scaled independently. The Flask API can then run on one or more application servers (container instances). Behind a load balancer, multiple app servers could handle concurrent requests from many users.
- *Worker Nodes:* If using a task queue for demo parsing, have one or more dedicated worker instances. These can be beefier machines optimized for CPU. The API server would enqueue parse jobs, and workers would dequeue and execute them (storing results in the DB).

- *Auto-Scaling:* For truly dynamic scaling, container orchestration like Kubernetes or AWS ECS could be used to automatically spawn more workers or API instances based on load. However, this adds DevOps complexity. As a solo dev, a simpler approach is to provision a slightly larger instance that can handle peak expected load, and scale vertically (upgrade the instance) as needed. Horizontal scaling (multiple servers) can be introduced once the load consistently approaches the single server's limits.

- **Serverless and Alternatives:** Using serverless functions (AWS Lambda, Google Cloud Functions) for the backend is *possible* for the API layer but not ideal for the parsing tasks:

- A serverless API could easily serve the Faceit stats (lightweight requests), but when it comes to parsing a demo (a 20s CPU task, potentially 300MB file), a Lambda might time out or incur high cost especially if run in parallel many times. Additionally, transferring large demo files in a stateless function environment is slow. A long-running container or VM is better suited for this workload.

- That said, one could use serverless for *auxiliary* tasks like a webhook or periodic trigger (e.g. a scheduled function to pre-fetch new matches every hour). The core demo analysis should remain on persistent servers for performance.

- **Containerization & DevOps:** The entire system should be containerized with Docker. This ensures the environment (Python version, libraries like demoinfocs/awpy, MongoDB, etc.) is consistent across dev and prod. The developer can use Docker Compose for local testing (one service for Flask, one for Mongo, one for a worker process). In production, those same images can run on the chosen host or be deployed to a container service. This approach simplifies deployment and makes scaling out easier if needed (just run more containers of the worker or API service).

- **Cost Management:** Initially, keep costs low by using a single host for everything. The tech stack (Flask, MongoDB, etc.) can all run on modest hardware when usage is limited. As paying users join, reinvest into better infrastructure. Using a managed database might add cost but offloads maintenance (backups, availability). The developer should monitor resource usage – CPU for parsing (which will spike during matchrooms) and memory for caching data – to know when to upgrade. Logging and monitoring (using something like CloudWatch, DataDog, or even simple custom logs) will help identify bottlenecks.

- **Reliability:** Even as a solo project, ensure basic reliability features:

- Automated backups of the database (if using MongoDB, enable daily backups or use a managed DB with backup).
- Use a reverse proxy (like Nginx or AWS ALB) in front of Flask for better performance and TLS termination. Flask can be run via Gunicorn or Uvicorn (if using an async framework) to handle multiple requests.
- Consider using a CDN or caching layer if any static assets or repeated requests (though most queries are user-specific and short-lived).

In essence, start simple on a VPS and evolve towards a cloud infrastructure as needed. A plausible deployment could be on **AWS**: e.g. an EC2 instance running Docker Compose for the MVP, then migrating to ECS/ECR with an Auto Scaling Group for the API, and maybe using AWS S3 for storing demo files (if we decide to cache many raw demos) instead of keeping them on disk. However, storing raw demos long-term may not be needed – processing on the fly or caching only extracted data is sufficient and lighter.

Given the developer's skill in Docker and back-end tech, maintaining a VPS or cloud instance is within reach. The hosting strategy should remain **flexible**: easy to maintain for one person and able to scale when the user demand (and revenue) justifies it. A **hybrid approach** can also be used – e.g. a cloud function could trigger a container-based workflow for parsing if trying to combine serverless triggers with container execution, but this is likely over-engineering for the scope. The recommended path is to **deploy on a cloud VM with Docker**, instrument it for monitoring, and gradually partition services (database, workers) as the system grows.

Chrome Extension UI/UX Design

Extension Integration with Faceit UI: The extension will inject a user interface seamlessly into the existing Faceit matchroom page. The goal is to present the rich data we gather in a clear, non-intrusive way that **augments** the match lobby without overwhelming the user. Here are the UI design suggestions: - **Sidebar Panel or Overlay:** A common approach is to create a collapsible sidebar or panel within the matchroom. For example, a panel on the right side of the screen could show a summary of opponent stats. It could be initially minimized and then expanded when the user clicks a "THE PRODUCT Insights" button or icon. This prevents cluttering the interface by default. The panel can use Faceit's styling (dark theme, similar fonts) so it feels native. Key information like map win rates could also be shown at a glance without even expanding (perhaps small icons next to each map). - **Inline Player Stats:** Next to each player's nickname in the matchroom lineup, the extension can insert brief stats. For instance, it might display a colored badge for their top 1-2 maps or a symbol indicating their role. One idea: color-code the win rate of the currently hovered map next to each player – if you hover "Inferno" during veto, each player's Inferno win% appears by their name (green for high, red for low). This contextual display helps quickly identify who is uncomfortable on that map. Extensions like VisusGG have implemented similar UI enhancements: VisusGG *"color-codes the statistics to let you analyze the visible data easily"* and even allows toggling comparison mode to see both teams' stats side by side [11]. We can draw from those ideas to make a **minimal UI that conveys key data at a glance**. - **Map Veto Assistance UI:** During the veto, the extension could highlight recommended actions. For example, if the opponents have a very low win rate on a particular map, mark that map with a thumbs-up or highlight as a good pick for your team. Conversely, if they are very strong on a map, highlight it as a ban suggestion. This could be done by overlaying icons or changing the outline of the map in the veto interface. The design should remain subtle (perhaps an icon that when hovered shows

the exact stat reasoning, e.g. “Opponents 30% win rate on this map”). - **Post-Veto (Pre-game) Details:** Once the map is decided and teams are locked in, the extension can switch to show map-specific opponent insights. This could be a small table or list in the sidebar: listing each opponent and bullet points of their tendencies (e.g. *“Plays B site, aggressive pushes”*, *“AWPer, anchors A”*). If possible, a mini overhead map image with heat zones might be shown for each player – but only if we can render it clearly and quickly. Another option is to have a tabbed interface: one tab for **“Map Stats”** and another for **“Players”**. During veto you focus on Map Stats, after veto you switch to Players to read about each opponent.

- **Clean and Readable Layout:** We will keep paragraphs of info short and use icons or tooltips to save space. Important numbers (like win percentages, KD ratios) will be bold or color-highlighted. We should avoid long text in the extension; instead use icons (grenade icon for utility info, a helmet icon for headshot %, etc.) with tooltip explanations. A small legend or the ability to toggle detail levels can help manage complexity (e.g. a “basic view” that just shows map win rates and a “detailed view” that expands each player’s profile).

- **Example UI Flow:**

- User opens matchroom – extension adds a widget (say a small logo or text like “Click for Match Insights”).
- User clicks it – a sidebar slides in showing a list of maps with team win rates and a suggested ban/pick. The user can quickly tell which map to aim for.
- After map is selected, the extension automatically updates the panel to “Opponent Analysis on [MapName]”. The user sees each opponent’s name with a few icons/stats next to it, and can hover or click a name to see a dropdown with more details (like their heatmap or preferred weapons).

- The extension could also have a comparison view – e.g. *Team A vs Team B average stats* – but this is secondary to the main goal of opponent scouting. Still, a small summary like “Your team average Elo vs Opponent team average Elo” or recent form can be shown to frame the match difficulty.

- **Performance and Responsiveness:** The UI should load quickly and not lag the page. All data computations happen in the backend, so the extension is just rendering received data. We should ensure the content script doesn’t overly frequently poll or manipulate the DOM in inefficient ways. Using modern frameworks can help structure the UI, but even vanilla JS with direct DOM manipulation might suffice since the UI isn’t extremely complex. The developer is comfortable with Vue.js, which could be used to build a reactive interface for the extension. For instance, the extension’s content script could be a Vue app mounted onto a div in the Faceit page. This would simplify dynamic updates (Vue can data-bind the stats we receive to the HTML elements). The trade-off is including a framework increases extension bundle size slightly, but 200KB or so (as seen in similar extensions [24]) is not a big issue. Many existing extensions successfully use frameworks to manage their UI.

- **Freemium UI Considerations:** If certain insights are premium-only, the UI should reflect that gracefully. Perhaps some sections or deeper details are shown blurred or with a lock icon, and a tooltip “Unlock with Premium”. The extension might include a login for the user to authenticate as a premium user (possibly via Faceit OAuth or a simple email login). That login flow can be an embedded popup or a redirect to our web service to sign in. This is additional UI flow to design but

can be kept simple for MVP (even a text field to enter an API key or token the user gets from our site could work).

In summary, the extension's UI will be **intuitive and minimal**, overlaying on Faceit's interface to provide at-a-glance info. We will use **logical grouping and toggles** so that users can quickly scan key points (maps, key player stats) without reading large blocks of text during the short pre-game period. By learning from existing tools – e.g. Repeek's in-match Elo display and VisusGG's map stat overlay – we ensure our UI enhancements genuinely help the player focus on strategy, not on deciphering data. The design will undergo iteration with user feedback, but the core idea is a slick overlay that delivers high-tier insights in a digestible format.

Tech Stack & Development Plan

Frontend (Chrome Extension): The extension will be built with standard web technologies (HTML/CSS/JavaScript). Given the developer's expertise, a modern JS framework like **Vue.js** can be used to structure the extension's UI for reactive updates. The extension will consist of content scripts that run on `faceit.com` matchroom pages, and possibly a background script for managing requests. Key components: - *Content Script:* Injects our app UI into the Faceit page and handles DOM interactions. This script will likely be written in JavaScript or TypeScript and bundled (using Webpack or a similar build tool) along with the Vue framework if used. It will listen for certain events (page loaded, or mutation observer to detect when matchroom info is present) and then trigger data fetch from the backend. - *Background/Service Worker:* This can manage persistent tasks like storing the user's authentication token (if login is required) and making cross-origin API calls to our backend (Chrome extensions can bypass CORS via background requests). Alternatively, the content script can make XHR/fetch calls directly to our API if CORS is enabled on the server. The background script might also handle things like extension icon clicks or any message-passing if needed between content and extension context. - *Manifest & Permissions:* We'll set the extension manifest v3, request permission to run on `https://*.faceit.com/*` URLs, and possibly `<all_urls>` if we fetch external resources. Also, if we use Faceit's official API directly (less likely, since we'll use our backend), we might need to include the API domain in permissions. However, the safer model is to route all data via our backend, so the extension only needs to contact our domain (which will be allowed in manifest). - *UI Libraries:* Vue.js (with its runtime ~80KB gzipped) is a reasonable choice. We can also use lightweight component libraries or icons (e.g. FontAwesome for icons like grenades, etc., or Faceit's own icon assets if available). For styling, simple CSS (possibly with a preprocessor like SCSS) will be used to match Faceit style. Ensuring our CSS is encapsulated (use unique prefix classes or Vue's scoped styles) will avoid conflicts with Faceit's CSS.

Backend: The backend stack will primarily be: - **Python + Flask:** Chosen for rapid development of RESTful APIs. Flask is lightweight and the developer is proficient with it. The API will have endpoints such as: - `/api/match-insight?match_id=XXX` or `/api/prematch?players=[id1,id2,...]` – returns the compiled insights. - `/api/login` and `/api/user` for handling user auth (if needed for premium). - Possibly endpoints for health checks, and internal use (e.g. a webhook endpoint if Faceit provided any event push – not sure if applicable, but maybe not needed). - **Data Processing:** Within the Flask app (or alongside it), we'll integrate the **demo parsing logic**. If using Awpy (Python library wrapping demoinfocs), the Flask app can directly call parsing functions in worker threads or subprocesses. For example, when a request for analysis comes in, Flask can offload the heavy parsing to a Celery worker or a thread and quickly return a response like "processing" which the extension can poll. But for MVP, it might be simpler to have the request hang for a couple seconds while parsing completes then return the data. Flask can handle that as long as

we don't hit request timeouts. - **Task Queue:** To better structure parsing jobs, we can use **Celery** (with Redis or RabbitMQ). The developer is familiar with MQTT, which is not directly used here, but that familiarity indicates comfort with messaging systems – so adopting Redis as a lightweight message broker for Celery is feasible. This would allow the API to enqueue a "parse these demos" task and immediately respond to the extension with whatever is ready (maybe basic stats), then the extension can poll a second endpoint for "is the analysis done?". This adds complexity, so it's optional. Another approach is synchronous for simplicity: the extension makes one call and waits for the full response. Given that parsing might be just a few seconds, this could be acceptable for now. - **Database: MongoDB** will be used to store persistent data: - Player analysis records (e.g. a collection `player_insights` keyed by player ID, containing their aggregated stats, recent performance metrics, heatmaps, etc.). - Match records (could store results of parsed matches if needed). - User accounts and subscription status if a login system is in place (alternatively, this could be a SQL DB, but Mongo can store user profiles too since it's simple data). - MongoDB is chosen because our data is inherently document-oriented (each player or match's stats as a JSON). It allows flexible schema – easy to add new metrics without migrations. The developer is also comfortable with MariaDB/SQL, which could be used for structured data like user subscriptions or if we needed complex relational queries (not likely for this use case). A hybrid could be used: SQL for user accounts, NoSQL for game data. But to keep things straightforward, one database (Mongo) can suffice for all, since the data volume is not huge and we can structure user data as needed. - **External Libraries:** We will use the Faceit Data API via HTTP requests (Python `requests` or an official Faceit SDK if available). For demo parsing, integrate **Awpy or demoinfocs** as discussed. We'll also use libraries for data manipulation (pandas or NumPy if doing any complex calculations, though likely not heavy) and maybe for generating any visuals (if we create heatmap images server-side, a library like Matplotlib or PIL could plot player positions on a map image). However, generating images on the fly might be too slow for matchroom timing, so we may instead send raw coordinates to the extension to draw or just send text summaries. For the MVP, focus on textual insights, maybe add visualization later. - **Testing:** Use Pytest for backend testing (especially for the data processing functions to ensure parsing is correct). The extension can be tested manually in Chrome, but also unit test some of its logic if structured well.

Tech Stack Justification: All chosen components align with the developer's skill set (Python, Flask, JS, Vue, Docker, SQL/NoSQL). The stack is also quite common for modern web applications, meaning plenty of community support and documentation. For example, a Flask+Mongo backend is a well-trodden path for data-driven applications, and using Vue in a browser extension is also a known pattern (there are templates for Vue-powered Chrome extensions).

Development Plan Outline: 1. **Setup Skeleton:** Initialize a Flask project with basic routes and a Vue project for the extension. Set up MongoDB (perhaps use Docker Compose to bring up a Mongo container for development). Test a simple end-to-end call: extension calls a test API endpoint and displays a result. 2. **Integrate Faceit API:** Write a module in the backend to communicate with Faceit's API (e.g. a Python class `FaceitClient` that can `get_player_stats(player_id)` and `get_match_history(player_id)` etc.). Test it with known player IDs and ensure we can retrieve `demo_url` links [6] and stats [7]. This requires obtaining a Faceit API key and handling rate limits (maybe add a simple caching layer in this module). 3. **Demo Parsing Pipeline:** Integrate Awpy or another parser. Start with a single demo file (perhaps download a sample .dem from Faceit) and write a parser function that extracts the desired info (e.g. list of kills with positions, grenade throws, etc.). This will be the most complex part – deciding what events to capture and how to summarize them into insights. We will likely iterate here: first get basic data working (like counting rounds, kills, etc.), then add specific analyses (heatmap calculation, etc.). We'll leverage demoinfocs' capabilities to get player positions and events via callbacks [25]. 4. **Data Aggregation:**

Build the logic to aggregate parsed data into the final stats for a player or team. For example, after parsing 5 demos for a player, combine all round positions to compute their most common locations, combine all matches to compute overall win rate on each map (as cross-check to Faceit's stat), etc. This could be implemented as a series of functions that produce sections of the insight JSON (one for map stats, one for playstyle, one for utility, etc.). 5. **Backend Performance Checks:** Test parsing multiple demos concurrently (simulate a match analysis). Optimize by adding caching: e.g. store results in Mongo and ensure the second time you request the same player, it returns from DB quickly. This step is to validate our earlier performance considerations. 6. **Frontend UI Implementation:** Develop the extension UI to display data. Use dummy data initially to design the layout (e.g. mock a JSON response and have the Vue app render it). Then connect it to real API calls. Implement the content script to detect match start and inject UI. Ensure it plays well with Faceit's SPA nature (Faceit website might be a single-page application, so when navigating to a matchroom, our extension needs to catch that route change – possibly using URL pattern matching in the extension manifest and a small delay to let the content load). 7. **Polish & UX:** Add loading indicators in the UI (so user knows data is being fetched/parsing happening). Handle error cases (if API fails or no data available for a player, show "No data" gracefully). Make the UI elements toggleable or collapsible as needed. Also, test on different screen resolutions to ensure the overlay doesn't break the underlying page layout. 8. **Security & Deployment:** Secure the backend (use HTTPS, require an API key or auth token from the extension to prevent abuse of our API by others). Possibly implement a simple auth where the extension includes the user's Faceit ID or our own token so we can identify premium users. Deploy the backend on the chosen server with Docker. Submit the extension to Chrome Web Store (which requires a thorough description of permissions and functionality).

Tech Stack Summary: - *Language:* Python (backend), JavaScript/TypeScript (frontend). - *Backend Framework:* Flask (with Flask-RESTful or FastAPI could also be considered for faster JSON handling, but Flask is sufficient). - *Database:* MongoDB for primary data. (MariaDB/SQLite could be added if needed for relational storage, but not mandatory). - *Data Processing:* Awpy (Python CS2 demo parser) under the hood using demoinfocs-golang for speed [17]. - *Frontend Framework:* Vue.js (for extension UI) plus plain DOM manipulation where straightforward. - *Deployment:* Docker containers on a cloud VM (with potential migration to managed services as needed). - *Libraries:* requests (Faceit API calls), Celery or threading (async tasks), numpy/pandas (if needed for calculations), and possibly small utility libs for things like heatmap generation or stats.

This stack is chosen to **maximize development speed and align with the developer's expertise**, while still being performant enough for the use case. By using proven libraries and services, we minimize reinventing the wheel. The end result will be a maintainable codebase where the heavy lifting is done in Python/Go and the user-facing part is a neat JavaScript extension – a combination well-suited to delivering real-time insights to players.

References:

1. Faceit Data API documentation for retrieving players, matches, and demo URLs [6] [4].
2. Scope.GG Prematch – confirms viability of downloading/analyzing last 5 matches for insights [8] [9].
3. Demoinfocs-golang – high-performance CS2 demo parsing library (benchmarked at ~25 min of gameplay parsed per second) [15], used by major services [16].
4. Andrew Healey on CS2 demos – highlights size (300MB+ demos, ~20s parse) and importance of caching results [21] [22].

5. Chrome Extensions (Repeek, VisusGG) – examples of enhanced Faceit UI via extension, showing map win rates and team Elo in matchrooms [3] [11].
6. Awpy – Python library wrapping CS2 demo parsing, built on the proven Go parser [17].

---

[1] [2] [25] **Rendering Counter-Strike Demos in the Browser — Andrew Healey**
https://healeycodes.com/rendering-counter-strike-demos-in-the-browser

[3] **Repeek (formerly FACEIT Enhancer) - Chrome Web Store**
https://chromewebstore.google.com/detail/repeek-formerly-faceit-en/mokknliiomknodkdmpcellamkopbdmao?hl=en

[4] [5] [7] **GitHub - kallelundgren93/faceit-ruby: This gem is for the data v4 api @faceit.**
https://github.com/kallelundgren93/faceit-ruby

[6] **Docs | FACEIT for Developers**
https://docs.faceit.com/docs/data-api/data

[8] [9] [13] **Pre-match FACEIT Analyzer for CS2 | SCOPE.GG**
https://scope.gg/prematch/

[10] **Retrieving FACEIT Data | FACEIT for Developers**
https://docs.faceit.com/getting-started/Guides/retreiving-faceit-data/

[11] [24] **VisusGG - Chrome Web Store**
https://chromewebstore.google.com/detail/visusgg/kodlabmmaalpolkfolgpahbjehalecki?hl=en-US

[12] **alexarne/Lookback-FACEIT - GitHub**
https://github.com/alexarne/Lookback-FACEIT

[14] [15] [16] [17] **GitHub - markus-wa/demoinfocs-golang: A Counter-Strike 2 Demo Parser for Go (demoinfo)**
https://github.com/markus-wa/demoinfocs-golang

[18] **GitHub - pnxenopoulos/awpy: Python library to parse, analyze and visualize Counter-Strike 2 data**
https://github.com/pnxenopoulos/awpy

[19] **GitHub - ValveSoftware/csgo-demoinfo: CS:GO demo parsing tool**
https://github.com/ValveSoftware/csgo-demoinfo

[20] **demoinfocs-golang - A CSGO Demo Parser - Go Packages**
https://pkg.go.dev/gopkg.in/markus-wa/demoinfocs-golang.v1

[21] [22] [23] **Compressing CS2 Demos — Andrew Healey**
https://healeycodes.com/compressing-cs2-demos